Guía de Laboratorio 7

Entrega Grupal

Objetivo

- El objetivo de este laboratorio es avanzar en su proyecto y resolver dudas.
- Trabajarán sobre su dominio propio.
- No es necesario que el sistema esté completo hoy, pero sí que compile y ejecute sin errores antes de entregar.
- Etiqueta de requisito: al completar cada tarea, agreguen un comentario `// [REQUISITO: X]` en el código. Facilita la revisión.
- Recuerde documentar todos sus supuestos de diseño y el porqué de su elección.

Tareas (10 pts c/u)

Tarea 1 · Static y Final

1.1 Identifiquen en su proyecto al menos dos constantes que hoy estén escritas como valores "a mano" (hardcoded). Ejemplos típicos:

- El nombre del archivo de datos ("inventario.dat", "productos.dat")
- Un límite de stock mínimo, un precio mínimo, una comisión
- El nombre del sistema o de la empresa

Declárenlas como static final en la clase que corresponda. Etiqueten con // [REQUISITO: Static y Final].

1.2 Identifiquen al menos un método utilitario que no use atributos de instancia (por ejemplo, validar un valor, calcular un descuento, formatear un precio). Conviértanlo a static. Etiqueten con // [REQUISITO: Static y Final].

1.3 Compilen con F6. El proyecto debe compilar sin errores.

Tarea 2 · Uso de excepciones propias

¿Qué es una excepción propia?

Java permite crear excepciones personalizadas para errores específicos del dominio del problema. Por ejemplo, en un sistema de inventario tiene más sentido lanzar StockInsuficienteException que un genérico IllegalArgumentException.

Una excepción propia:

- Extiende RuntimeException (no necesita declararse con throws en los métodos)
- Recibe información de contexto en el constructor y la pasa a super(mensaje)
- Se lanza con throw new MiExcepcion(...) en el método donde ocurre el error
- Se captura con try-catch en el punto de uso — generalmente el menú

Patrón: cómo crear una excepción propia

```
// Paso 1: Crear la clase de excepción
class StockInsuficienteException extends RuntimeException {
    public StockInsuficienteException(int actual, int pedido) {
        super("Stock insuficiente: hay " + actual
              + ", se pidieron " + pedido + ".");
    }
}
```

```java
// Paso 2: Lanzarla en el método donde ocurre el error
public void vender(int cantidad) {
    if (cantidad > stock) {
        throw new StockInsuficienteException(stock, cantidad);
    }
    stock -= cantidad;
}
```

```java
// Paso 3: Capturarla en el menú con try-catch
try {
    gestor.vender(cantidad);
} catch (StockInsuficienteException e) {
    System.out.println("Error: " + e.getMessage());
}
```

2.1 Identifiquen en su proyecto al menos dos situaciones de error propias del dominio.

2.2 Creen una clase por cada excepción siguiendo el patrón de arriba.

2.3 Lancen la excepción con throw new ... en el modelo o en la clase gestora. Validen también en los constructores.

2.4 Captúrenlas con try-catch en el menú. Muestren un mensaje claro al usuario — nunca el stack trace.

2.5 Etiqueten con // [REQUISITO: Excepciones] las clases de excepción y los puntos de throw.

2.6 Prueben deliberadamente el error desde el menú para confirmar que se captura correctamente.

Tarea 3 · Persistencia

3.1 Agreguen implements Serializable y opcionalmente private static final long serialVersionUID = 1L a todas las clases de modelo. Etiqueten con // [REQUISITO: Persistencia].

3.2 Implementen guardar() en la clase gestora usando ObjectOutputStream. Etiqueten con // [REQUISITO: Persistencia].

3.3 Implementen cargar() usando ObjectInputStream. Verifiquen que el archivo existe y no está vacío antes de abrirlo.

3.4 Integren en main(): gestor.cargar() al inicio del programa, gestor.guardar() en case 0 al salir. No como opción de menú.

3.5 Prueba: agreguen 3 registros → cierren el programa → vuelvan a ejecutar → verifiquen que los datos persisten.

Recuerden: cargar() al arrancar, guardar() al salir (case 0). El usuario no debería tener que hacerlo manualmente.

Tarea 4 · Persistencia

4.1 Para cada clase modificada, verifiquen que el diagrama UML muestre:

- Los atributos static final (subrayados en UML)
- Los métodos static (subrayados en UML)
- Las clases, métodos y atributos que aún no haya incluido y sean parte de su diseño.

4.2 El diagrama debe estar coherente con el código actual.

Importante: Solo un integrante debe entregar el laboratorio.

- Formato nombre archivo: “matricula1_matricula2_matricula3.zip”